Project 2 Report — Mise en Place

Administrative

Team name: NaN of Your Business

Members (GitHub usernames): Samuel Deegan (samueldeegan), Will O'Flaherty, Laveena Bhuiyan

GitHub repository: https://github.com/samueldeegan/llm-benchmark-contamination-detector

Video demo: https://youtu.be/rsBqola8Nkc

Extended and refined proposal

Problem

Given a training corpus and a benchmark, find every benchmark question with at least one 13-word stretch that appears word for word in the corpus, after both are cleaned the same way. The project compares two from-scratch structures that answer the same question: is this 13-word sequence in the corpus?

Motivation

Language model performance is evaluated using benchmarks such as Massive Multitask Language Understanding (MMLU). If benchmark questions leak into a model's training text, a high score can reflect memorization rather than ability. This problem is called contamination. The original GPT-3 paper demonstrated a method for detecting contamination by identifying 13-word overlaps between evaluation questions and the training set (Brown et al., 2020). Our project borrows the 13-word comparison window from their design. We planned to implement this detection regimen using a hash table and a suffix array; however, team issues led to only the hash table method being implemented. In this project we chose a large training set called AG News, a collection of news articles from over 2000 sources, and checked to ensure that it did not contain any MMLU questions on its own. We chose a set without any contamination because we wanted to be able to assess our algorithm's detection rate using 100 planted questions in the corpus.

Features implemented

- A CSV reader that uses commas, line breaks and quoted fields to parse the file text into separate fields and rows within a nested vector. Another function joins the chosen columns of each row of both the MMLU and AG News into a unified string.
- Text processing module that replaces non-alphanumeric characters with spaces and converts all capital letters to lowercase in order to create a universal word format. Additionally, it creates an unordered map that translates words into sequentially calculated ID numbers. It then converts the words in the training set and question text to IDs and stores them into corresponding vectors.
- A question planting module which, using a fixed seed, randomly selects 100 questions which are greater than or equal to 13 words in length from the MMLU set, then inserts them at the end of 100 randomly chosen articles within the AG News set. It then stores an answer key that is used to score detection rates later on. In addition, it counts how many questions in the MMLU were shorter than the size of our comparison window.
- A vector-based hash table implementation that is two times the size of the number of 13-word windows that do not cross between distinct articles in the entire AG News corpus. Then each window in the corpus is hashed, and the corpus' index of the first word in the window is stored in its respective table slot. If the slot at a window's computed hash is already taken, then the successive slots are checked using linear probing.
- A detection module called question checker which iterates through all questions in the MMLU, skips ones which are shorter than 13 words, then iteratively checks the hash table for identical entries to each of the 13-word windows in that particular question until it either detects a matched window or has exhausted all comparison windows.

- A module called measurement that uses chrono to measure the wall-clock time of the hash table build and the search for matching questions. This is done 5 times (or as otherwise specified) and the median result is printed after running the program. Separately, the program also reports the memory usage of the hash table, which is calculated in bytes by multiplying the number of slots by 4.
- A module that checks the detection algorithms' correctness against the answer key created in the planting module. In addition, it also lists the questions which were detected using the method but not specifically planted. (more on this below)
- A command-line program that runs everything, with options for columns, headers, plant count, seed, and runs.

Not implemented: the suffix array and the GUI.

Description of data

Corpus: AG News training set: 120,000 news articles in a CSV with class, title, and description columns and no header row. Each document is the title plus the description. After cleanup and planting, the corpus has 4,765,177 words, 65,122 of them distinct.

Benchmark: MMLU test set (Hendrycks et al., 2021): 14,042 questions in 57 subject files, joined alphabetically into one CSV. Only the question column is used.

Tools, languages, and libraries

C++17 with g++ (-O2), GitHub Codespaces, and Git/GitHub. C++ standard library only: vector, string, unordered_map, random, algorithm, chrono, fstream, sstream, memory.

Algorithms implemented

Hash table with linear probing (src/structures/hash_table.cpp). The keys are the 3,325,177 corpus windows of 13 words that don't cross a document boundary, each stored as its starting position.

- **Hash:** $h = h \times 31 + id$ over the 13 word IDs, in a 64-bit unsigned integer, modulo the table size.
- **Table:** a flat array of $2W + 1$ slots (W = number of windows), so it stays under half full. Each slot is 4 bytes: a corpus position, or -1 for empty.
- **Build:** hash each window and probe forward, wrapping at the end, to the first empty slot. A slot already holding the same 13 words means a duplicate, so the window is skipped.
- **Contains:** Reads the window directly from the question's word list. It returns false immediately if any constituent word ID is -2 (indicating it does not exist in the corpus). Otherwise, it probes the table: returning true when a slot's 13 corpus words match the window exactly, and false upon reaching an empty slot. Comparing full word sequences explicitly prevents hash collisions from producing false positives.

Suffix array (designed, not implemented): merge-sort every corpus position by the words that follow it, then answer lookups with binary search.

Additional data structures and algorithms

A shared corpus array (vector<int> of word IDs, -1 after each document) read by pointer and never copied; an unordered_map word table for creating word IDs; a base class Structure with virtual build, contains, and bytesUsed, so the checker and timing code work with either the hash table or suffix array; a CSV parser; std::shuffle with mt19937 (seed 42) for planting; and sorting for the median.

Distribution of responsibility and roles

The proposal gave the GUI and integration to Will, and one structure each to Sam and Laveena.

- **Samuel Deegan:** project plan and design documents, data pipeline, hash table, question checker, measurement, correctness check, and command-line program.
- **Will O'Flaherty:** did not contribute to the project.
- **Laveena Bhuiyan:** did not contribute to the project.

Analysis

Changes made after the proposal, and why

Change	Why
The suffix array and GUI were not implemented.	Neither Laveena nor Will participated in the building of the project.
Sam also wrote the pipeline, checker, measurement, correctness check, and command-line program.	The proposal was barebones and did not specify the modules that would be necessary in order to create the desired output. These modules were required in order to meet our original goal.
A command-line program was added.	It runs everything in the codespace, produces the report's numbers, and doesn't depend on the GUI. GUI was never delivered by Will.
Removed the Safe / Unsafe (from contamination) data set classification from the program output.	Every run plants 100 questions, so every run would be "Unsafe." This changed largely as a consequence of reevaluating our initial proposal.
The correctness check uses the answer key only, rather than consensus amongst the output of different structures.	Checking that both structures agree needs the second structure, which was never provided by Laveena.

Big O worst-case time of the major functions

c = characters in a file, w = corpus words, V = distinct words, N = documents, Q = questions, F = flagged questions, W = corpus windows, Wq = question windows (510,705), n = corpus positions, R = timing runs.

Function	Worst case	Average case
readCsv, extractText, cleanText, splitWords	$O(c)$	$O(c)$
buildCorpus, questionsToIds	$O(c + w \cdot V)$	$O(c)$
pickQuestionsToPlant, plantQuestions	$O(c + Q + N)$	$O(c + Q + N)$
HashTable::build	$O(W^2)$	$O(W)$
HashTable::contains	$O(W)$	$O(1)$
checkQuestions	$O(Wq \cdot W)$	$O(Wq)$
checkCorrectness	$O(Q + F)$	$O(Q + F)$
median	$O(R \log R)$	$O(R \log R)$
Suffix array build (designed)	$O(n^2 \log n)$	$O(n \log n)$ comparisons
Suffix array contains (designed)	$O(\log n)$	$O(\log n)$

The hash table's worst-case time complexity occurs when all windows hash to the exact same slot. However, keeping the table under half full maintains short average probe lengths, with each probe performing at most 13 word comparisons. Similarly, the worst case for the word table occurs when every word maps to a single unordered_map bucket.

Results

One run of the command-line program in the codespace, built with -O2. Times are medians of 5 runs.

Measure	Hash table
Build time	367 ms
Lookup time, all 14,042 questions	61 ms (about 4.3 µs per question)
Memory	26,601,420 bytes (6,650,355 slots × 4 bytes)
Questions flagged	994
Planted found	100 of 100
Unplanted questions flagged	894

Memory leaves out the corpus array (about 19.5 MB), which both structures share. The build takes about 6 times as long as checking every question.

Due to the lack of participation of group members, the results for the suffix array were never calculated.

Why 894 unplanted questions were flagged

As stated at the beginning, we checked to ensure there was not contamination in the AG News data set prior to planting the questions ourselves. These anomalous flagged questions are the result of many of the questions in the MMLU sharing the same structure. For instance, they may share the first few sentences and then vary in the last one. This in combination with our 13-word comparison window results in questions being marked as detected when it's just that the planted question and the flagged one share some text.

Reflection

Overall experience as a group

The group never worked as a team. Laveena did not take part in the proposal or the project, and Will, who had been in an accident, started late in the final week. When I mapped out the full program on Nov 3 and 4, I found that the proposal assigned only the two data structures and the GUI, and left the data pipeline, question checker, measurement, and correctness check unassigned. Will saw my plan to divide that work as expanding the scope, and he built his own version from the proposal to submit separately. I contacted the instructor on Nov 3. He told me to finish my own portion, and later removed me from the group so it could be graded on its own. My own communication was also late: I raised the unassigned work only days before the deadline.

Challenges

Besides planning around uncertain teammates, the challenges came from the real data and from the hash table. The MMLU file has 18,738 lines but only 14,042 questions, because some questions contain line breaks inside quotes, so the CSV reader has to keep track of quotes. The hash overflowed an int until I switched to a 64-bit integer, and the probe loop needed a correct stopping rule. Tracing the 894 indirect matches back to text that MMLU questions share also took time.

What we would change if starting again

I would start earlier and map out the whole program before writing the proposal, so that every part has an owner. I would also set check-in dates and agree on the shared interface in the first week so parts could be built separately.

What each member learned

- **Samuel Deegan:** parsing CSV files with quoted fields, hashing word sequences with a polynomial hash, resolving collisions with linear probing, sharing one interface between structures with a virtual base class, and timing code with medians. I also learned why 13-word

matching flags shared boilerplate. Without a second structure, I missed the comparison I most wanted to learn from.

- **Will O'Flaherty:** worked separately after the group split, so I can't speak to what he learned.
- **Laveena Bhuiyan:** did not take part, so I can't comment.

References

1. Brown, T. B., et al. (2020). Language models are few-shot learners. *NeurIPS 33*. arXiv:2005.14165.
2. Hendrycks, D., et al. (2021). Measuring massive multitask language understanding. *ICLR 2021*. arXiv:2009.03300. Data: https://people.eecs.berkeley.edu/~hendrycks/data.tar

AI use. Claude (Anthropic), model claude-opus-5-5, through claude.ai. Used to explain concepts (CSV parsing, hashing, linear probing, pointers, virtual classes, timing); to review code Sam wrote and point out errors, which he then fixed himself; to write the planning documents in 1. project-plan/; to check facts about the data files; and to draft this report. Example prompts: "help me understand the exact logic" (CSV reader); "check my work" (hash_table.cpp); "can you write up the [first] draft" (this report).